Project Design Phase-II
Solution Requirements (Functional & Non-functional)

Date	19 July 2026
Project Name	RentEase Platform
Maximum Marks	4 Marks

Functional Requirements:

Following are the functional requirements of the proposed solution.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	User Registration & Authentication	• Registration via Email and Password • Role Assignment (Renter vs. Owner) • Session authentication via JWT payload
FR-2	Admin Verification Portal	• View list of unapproved property owners • 1-Click manual verification of landlord accounts • Read-only auditing logs of platform activity
FR-3	Property Management (Owner)	• "Awaiting Verification" gate logic for unapproved owners • Listing Composer to upload property images, type, and rent amount • Listing Controller to toggle active lease states or delete postings
FR-4	Property Discovery (Renter)	• Dynamic Explorer Grid for visual housing gallery • Live text filtering across locations, types, and price limits • View detailed property specifications (furnishing, address)
FR-5	Tour Booking Coordinator	• Inline scheduler for Renter to submit visit date/time • Lease Scheduler for Owner to "Approve" or "Cancel" requests • Real-time visual status dashboard (Pending, Confirmed, Cancelled)

Non-functional Requirements:

Following are the non-functional requirements of the proposed solution.

NFR No.	Non-Functional Requirement	Description
NFR-1	**Usability**	Clean, distinctive visual aesthetic using Off-White & Deep Charcoal themes (Tailwind CSS). Incorporates subtle micro-interactions and scale transitions for clear feedback during navigation.
NFR-2	**Security**	Stateful session security strictly managed via cryptographically signed JSON Web Tokens (JWT). API routes are shielded by Express authorization middleware.
NFR-3	**Reliability**	Unified server-asset pipeline compiling static React assets via Vite/esbuild ensures stable delivery and smooth fallback routing to index.html for client-side navigation.
NFR-4	**Performance**	Fast Single Page Application (SPA) state updates minimizing page reloads. The lightweight JSON database (db.json) ensures rapid, low-latency data transaction handling.
NFR-5	**Availability**	Consistent server uptime via Node.js Express.js server on a unified port mapping (port 3000), reducing the complexity of running separate web and API servers.
NFR-6	**Scalability**	RESTful API architecture cleanly separates the frontend from business logic, allowing easy future migration from db.json to a robust RDBMS/NoSQL database without altering the client.